

# Designing a Simple 3D Block Jumper Game



This is a tutorial on creating a block jumper game in 3D graphics programming using the first person perspective. This follows up on the previous article about creating a block jumper game in the third person perspective, which was carried in the October 2018 issue of *OSFY*.

In this article, we shall move on to current times and recreate our previous game, but in first person perspective. We will achieve this by using a different perspective and by manipulating the camera to give an illusion of being in the 3D world.

For this project, we will not use any extra libraries but will solely use native Processing commands.

**Note:** We will use Processing (Python) with OOP concepts for most of the code; so if you are unsure of OOP in Python, I urge you to clear your concepts before beginning this tutorial.

## Setting up the basics

To start off, let's first create the screen on which we will draw the entire game. Unlike in the previous article, however, we must also define that the rendering is in 3D while defining the

size of the screen; and we achieve this by passing in an extra argument to the size function, P3D. This makes Processing recognise that we want 3D rendering, which will allow us to manipulate crucial aspects such as the perspective and the camera.

```
size(600, 600, P3D)
```

**Note:** You can find the entire code at the end of this article.

## Defining the path

We must next decide how the world will look to the player in the 2D game. Preferably, it will be blocks coming straight at the player. Now, to interest the player more, we have to define the path or road on which the player runs and jumps over blocks. To create this path, we will use a cuboid that extends along the Z axis set almost at

the bottom of the screen. We define this value in *setup()*, as given below:

```
base_ht = height-100
```

To draw this path, we use the following code in *draw()*:

```
background(137, 207, 240)
fov = PI/3
cameraZ = (height/2.0) /
tan(fov/2.0)
perspective(fov, float(width/2)/
float(height/2), 50, -cameraZ*20)

pushMatrix()
translate(width/2, base_ht, 0)
fill(0)
box(width/1.5, height/6, 500*100)
popMatrix()
```

Now, as you see, making a realistic cuboid is quite different from drawing